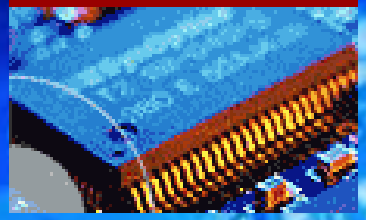
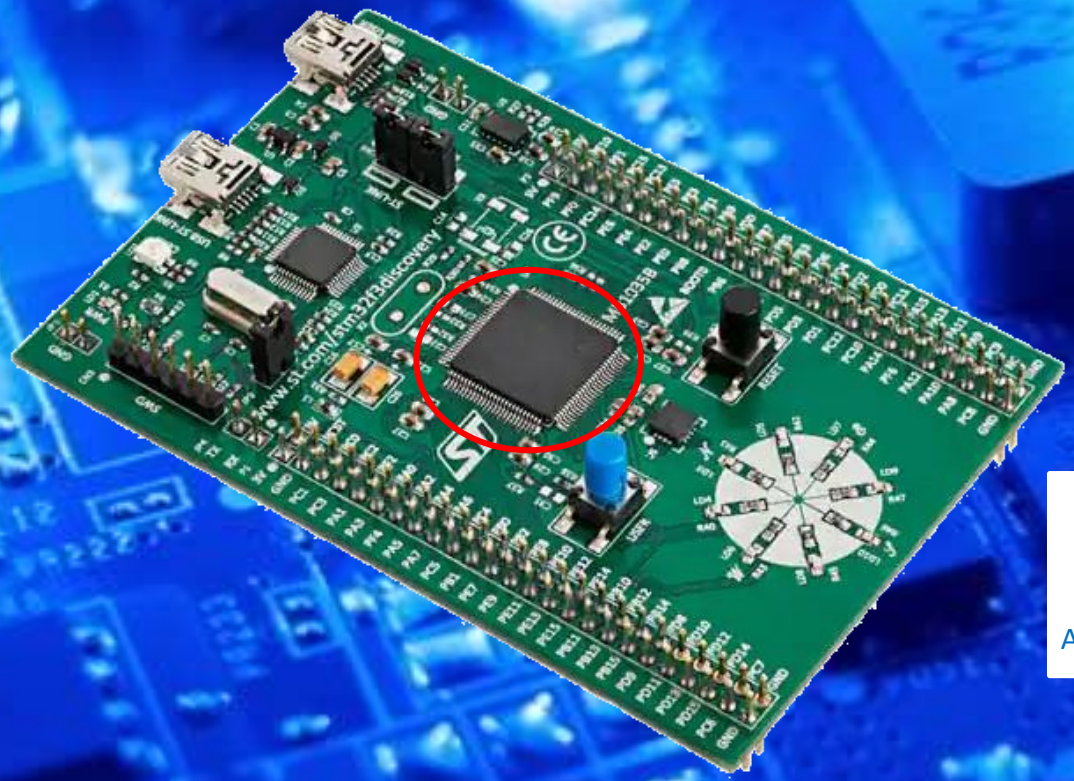
EEE3096S



Embedded Systems II



ARM
Advanced RISC Machines

Part1

ARM Assembly Programming 1/3

Embedded Systems II

L08

Dr Simon Winberg

Electrical Engineering
University of Cape Town

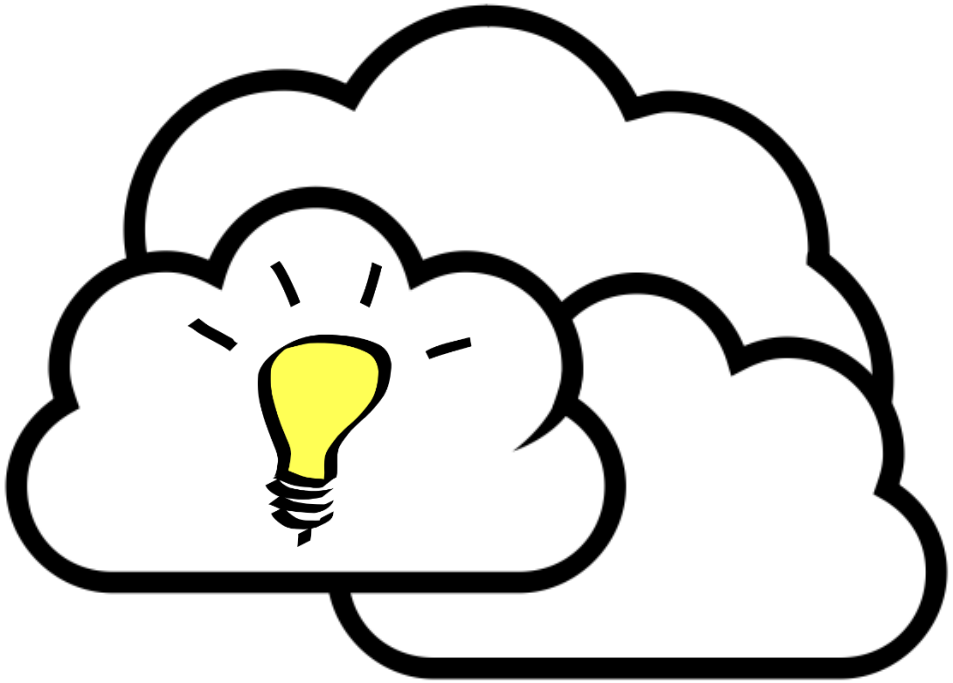
Outline of Lecture

- Assembly thinking activity
- Introduction to the ARM instruction set
- GNU ASsembler (GAS) Syntax
- Conditional Execution Instructions
- Data processing Instructions
- Load and Store

ARM

Advanced RISC Machines

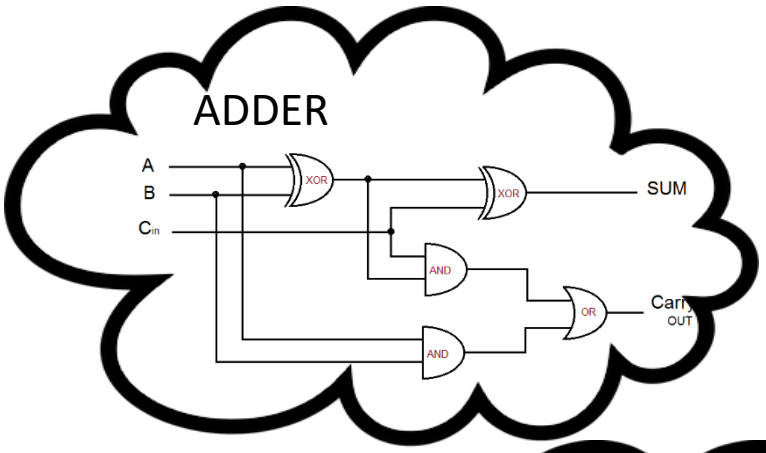
Thinking Activity

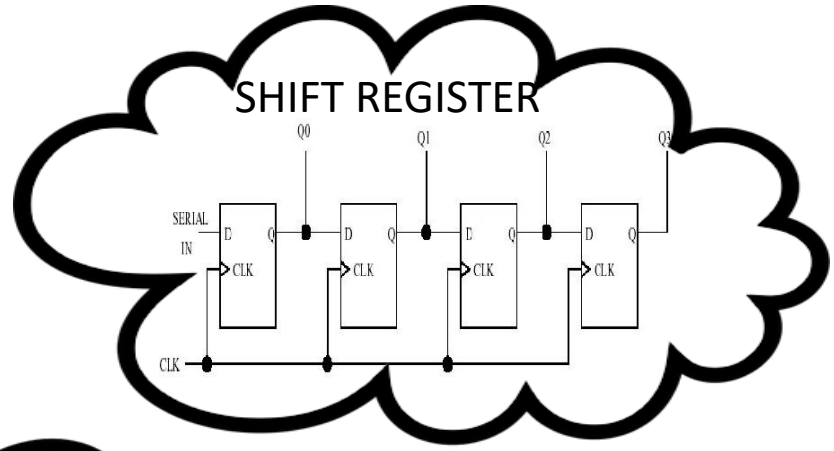


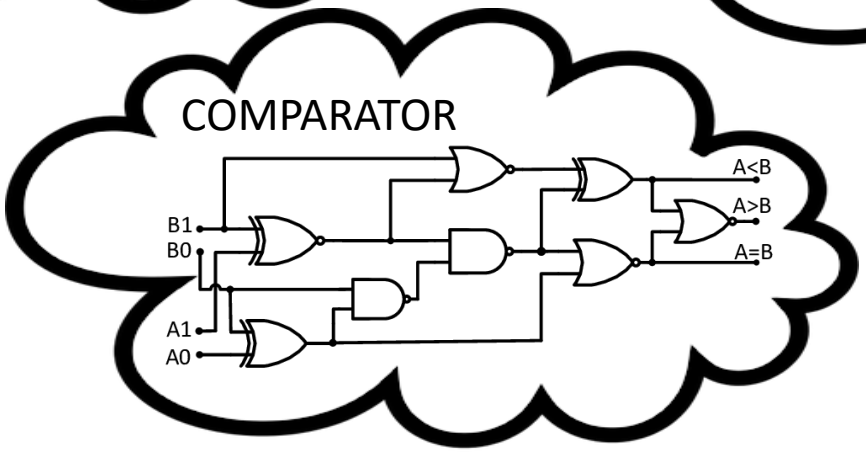
Pseudo Assembly Coding

Consider a CPU and its instructions

- Reflect back on digital logic experience
- You mind might be sparking ideas of e.g. …







CPU instructions

- These are the building blocks of CPU instructions
- This learning assembly using pseudo assembly approach is a two pass process…

Here is the first pass:

Todo: Thinking about instructions to use, and how to order them, to run a program.

(We haven't defined the instructions! But that's the point: think up your own ideas for them and their sequencing – that's the approach)

Here's the program that we want to run …

Program Description

We want to carry out the following processing:

```
int a, b, avg, res;
a = 100;
b = 200;
avg = (a+b)/2;
if (a>avg)  res=1;  // a grater than avg
if (a<avg)  res=-1; // a less than avg
if (a==avg) res=0;  // a same as avg
```

Don't bother with that yet!

View this C module **compaavg.c** in CODE.zip resources for this lecture (the completed ARM assembly version, compaavg.s, is included, don't look at that yet!)

Solution *

** Aiming to talk-through this, not so much as class activity (to save time)*

Program wanted:

```
int a, b, avg, res;
a = 100;
b = 200;
avg = (a+b)/2;
if (a>avg)  res=1;  // a grater than avg
if (a<avg)  res=-1; // a less than avg
if (a==avg) res=0;  // a same as avg
```

0. Some registers

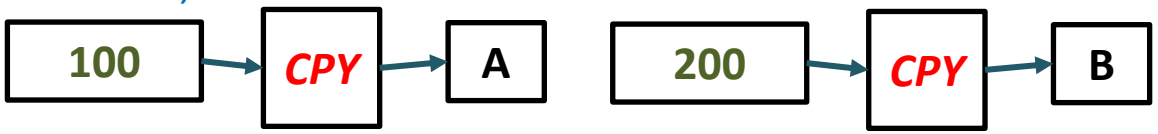
1. Assign these registers to values

2. Assign avg to (a+b)/2

3. Compare & do conditional assignment

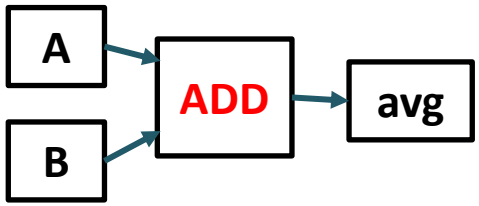
0. Some registers, let's say 32-bit ones

1. CPY A,100
   CPY B,200

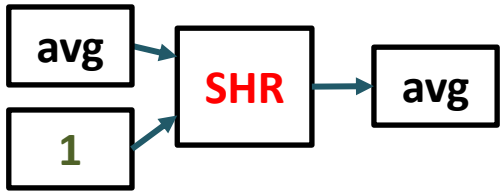


This is assembly pseudocode, not real ARM assembly. This done as a quick first attempt to avoid fussing with a specific syntax.

2. ADD AVG,A,B



2. SHR AVG, AVG, 1 // i.e. shift right is /2 for ints



3. CMP … etc …

// left as homework ☺

Thinking Activity Done

First pass of learning assembly: making up your own realistic assembly commands.

Hopefully you know the general approach now of writing assembly (and may be convinced also why this approach is useful for designing processors)

Now for second pass of learning ARM assembly specifically…

Recommended learning resources

- The following YouTube covers the essentials of assembly programming, and running the GCC assembler on a PC:

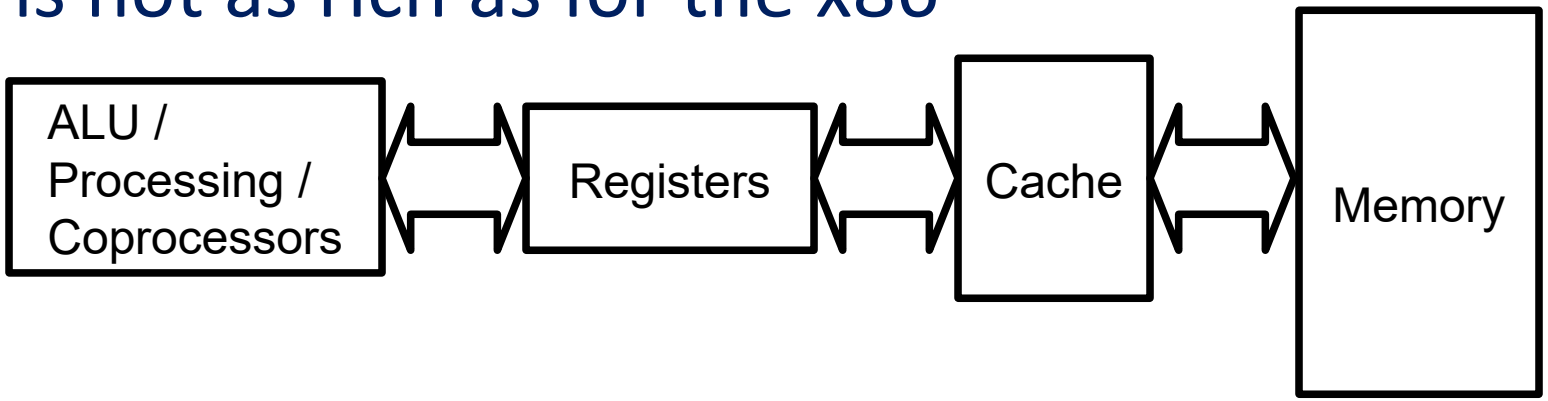
  https://www.youtube.com/watch?v=FV6P5eRmMh8

ARM Assembly Language Programming

ARM Data and Instructions

recap

- We will focus on the ARM AArch32 state
  (see lecture L07 for alternate states)
- Instructions and data are 32bit words
- ARM uses Load/Store architecture – you cannot manipulate memory directly
- ARM is a RISC processor – the instruction set is not as rich as for the x86



ARM Instruction Set

Instruction set divided into six types:

1. Branch Instructions
2. Data Processing instructions
3. Status Register Transfer Instructions
4. Load and Store Instructions
5. Coprocessor Instructions
6. Exception-generating Instructions

ARM Instruction Set

Branch

B	Branch
BX	Branch and Exchange
BL	Branch and Link

Data Processing

ADD	Add
SUB	Subtract
RSB	Reverse Subtract
CMP	Compare
TST	Test
AND	Logical AND
EOR	Logical Exclusive OR
MUL	Multiply
SMULL	Sign Long Multiply
SMLAL	Signed Long Multiply Accumulate
MOV	Move
SWP	Swap Word
MVN	Move Not
ADC	Add with Carry
SBC	Subtract with Carry
RSC	Reverse Subtract with Carry
CMN	Compare Negated
TEQ	Test Equivalence
BIC	Bit Clear
ORR	Logical (inclusive) OR
MLA	Multiply Accumulate
UMULL	Unsigned Long Multiply
UMLAL	Unsigned Long Multiply Accumulate
SWPB	Swap Byte

Status-register transfer functions

MSR	Move to Status Register
MRS	Move From Status Register

Load and Store

LDR	Load Word
LDRSH	Load Signed Halfword
LDRSB	Load Signed Byte
LDRH	Load Half Word
LDRB	Load Byte
LDRBT	Load Register Byte with Translation
LDRT	Load Register with Translation
LDM	Load Multiple
STR	Store Word
STRH	Store Half Word
STRB	Store Byte
STRBT	Store Register Byte with Translation
STRT	Store Register with Translation
STM	Store Multiple

Co-Processor

LDC	Load To Coprocessor
CDP	Coprocessor Data Processing
MRC	Move From Coprocessor
STC	Store From Coprocessor

Exception-generating

SWI	Software Interrupt

GAS Syntax

GAS is the GNU Assembler, which comes with GNU **binutils** for a variety of platforms

Syntax:

[label:] ***<instruction>*** ***<suffix>*** *[operands]*

Notes:

- label is optional – only needed if you want to refer to the instruction
- Not all instructions have operands
- Comments start with @

Conditional Execution (AArch32)

- Conditional execution reduces the need for branches and increases code density
- Many ARM instructions can be executed conditionally
- Each instruction has a 4-bit condition code field
- We will be focusing on AArch32 instructions, these are documented at: https://static.docs.arm.com/100076/0100/arm_instruction_set_reference_guide_100076_0100_00_en.pdf

For full details on the ARM Cortex-35A processor, consult its Technical Reference Manual (available from https://developer.arm.com/documentation/100236/0002)

Conditional Execution

Mnemonic	Condition	CPU condition flags
EQ	EQual	Z set
NE	Not Equal	Z clear
CS	Carry Set/unsigned Higher or Same	C set
CC	Carry Clear/unsigned LOwer than	C clear
MI	Negative (MInus)	N set
PL	Positive (PLus)	N clear
VS	oVerflow Set	V set
VC	oVerflow Clear	V clear
HI	HIgher unsigned	C set and Z clear
LS	Lower or Same unsigned	C clear or Z set
GE	Greater than or Equal to	(N and V) set or (Nand V) clear
LT	Less Than	(N set and V clear) or (N clear and V set)
GT	Greater Than	((N and V) set or clear) and Z clear
LE	Less Than or equal to	(N set and V clear) or (N clear and V set) or Z set

Addition of Q flag in 64-bit state which indicates floating point saturation

Branch Instructions

B : Branch

- B <address> (eg B main)
- Address relative to PC +/- 32Mbytes
- Does not return

BL : Branch and Link

- BL <subroutine> (eg BL sqrt)
- Program counter is put into the link register
- To return *mov pc, lr* (note GAS operand order)

Conditional Branching

- Branches can be conditional
- The assembler equivalent of an *if* statement

B<suffix> <offset>

- Branch if suffix matches state of flag in the CPSR
- Examples:
  - **BEQ** : Branch if equal or zero
  - **BMI** : Branch if the result is negative
  - **BVS** : Branch if an overflow occurred

Conditional Branching

Branch	Interpretation	Normal uses
B	Unconditional	Always take this branch
BAL	Always	Always take this branch
BEQ	Equal	Comparison equal or zero result
BNE	Not equal	Comparison not equal or non-zero result
BPL	Plus	Result positive or zero
BMI	Minus	Result minus or negative
BCC	Carry clear	Arithmetic operation did not give carry-out
BLO	Lower	Unsigned comparison gave lower
BCS	Carry set	Arithmetic operation gave carry-out
BHS	Higher or same	Unsigned comparison gave higher or same
BVC	Overflow clear	Signed integer operation; no overflow occurred
BVS	Overflow set	Signed integer operation; overflow occurred
BGT	Greater than	Signed integer comparison gave greater than
BGE	Greater or equal	Signed integer comparison gave greater or equal
BLT	Less than	Signed integer comparison gave less than
BLE	Less or equal	Signed integer comparison gave less than or equal
BHI	Higher	Unsigned comparison gave higher
BLS	Lower or same	Unsigned comparison gave lower or same

Data Processing Instructions

- Syntax **OP<suffix>** <result>, <op1>, <op2>

ADD : Add without carry

- result = op1 + op2

ADC : Add with carry

- result = op1 + op2 + carry

SUB : Subtract without carry

- result = op1 - op2

Data Processing Instructions

- If you want to use flags, you must specify that the status register must be updated
- For example, to add two multi-word values:

```
ADDS R0, R4, R8  @Add without carry
ADCS R1, R5, R9  @ Add with carry
ADCS R2, R6, R10
ADC   R3, R7, R11 @No need to update flags here
```

Data Processing Instructions

- **AND**, **ORR**, **EOR** and **BIC** work similarly to **ADD**

For assignment, use **MOV**

- **MOV** r1, r2 @ r1 = r2 (assignment)
- **MVN** r1, r2 @ r1 = ~r2 (assignment with complement)

For comparison, use **CMP**

- **CMP** r1, r2 @ set condition flag on r1 – r2
- Same as **SUBS** but result is discarded

Data Processing: Shifting

- A shift can be applied to the second source register
- Can shift by a constant or by a register
- **LSR** logical shift left (**LSL** for left)
- **RRX** rotate right with carry (no left)
- **ROR** rotate right without carry (no left)
- **ASR** arithmetic shift right (no left)

e.g.:

ADD r1, r2, r3, LSL r4 @ r1 = r2 + r3*2^r4

Status Register Transfer

Content of registers can be moved into and out of the status registers using MSR:

MSR Move to status register

- **MSR<suffix>** <sr>, <reg>

e.g. **MSR** cpsr, r10

MRS Move from status register

- **MRS<suffix>** <reg>, <sr>

Load and Store

Content of registers are moved from and to memory using LDR and STR respectively:

LDR Load Register

- **LDR** r1, [r2] @ r1 = mem[r2]

STR Store Register

- **STR** r1, [r3] @ mem[r3] = r1

Index Addressing

- Index addressing can make load / store operations more efficient, i.e. it adds an offset to the base address as part of the same instruction.

Example of a C statements that would translate to these instructions.
Assume: int arry [100]; where each element of arry is 4 bytes.

Pre-index addressing

- **LDR** r1, [r2, #4] @ r1 = mem[r2+4]

int x = arry[i+1];

Auto Pre-index addressing

- **LDR** r1, [r2, #4]! @ r1 = mem[r2+4]; r2 += 4

int x = arry[++i];

Auto Post-index addressing

- **LDR** r1, [r2], #4 @ r1 = mem[r2]; r2+= 4

int x = arry[i++];

Load and Store of a 32-bit constant

GAS gives macros for easier loading and storing

Example:

LDR r14, =*addr* @ Load value at address *addr*

But how? You can't put a 32bit immediate (constant) into a 32bit instruction!

The '**=**' macro allocates a word in mem and puts the 32-bit address there, i.e. it translates to:

```
LDR r14, =0xFFFE0100

Translates into:

LDR r14, [pc, #0x20]

…

lbl_0xFFFE0100: .dw 0xFFE0100 @ 32-bit addr after the LDR
```

The Next Episode…

Lecture L09

- Using ARM Assembly Programming 2/3:
  - ARM Assembly Activity
- Using ARM Assembly Programming 3/3:
  - interfacing C and ARM assembly
  - program startup, runtime library, linking